

**Douglas Makey Mendez Molero**

Posted on Jul 15, 2019 • Edited on Jul 6, 2024



23



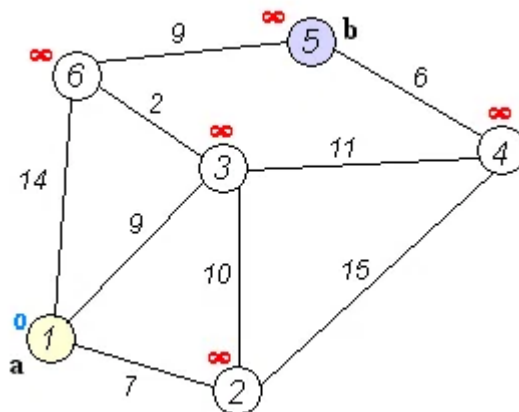
5

Implementation of Dijkstra using heap in Go

#go #dijkstra #heap #dev

Simple implementation of Dijkstra using heap in Go.

What is Dijkstra?



MEGA SHORT DESCRIPTION: Dijkstra's algorithm to find the shortest path between a and b. It picks the unvisited node with the lowest distance, calculates the distance through it to each unvisited neighbor, and updates the neighbor's distance if smaller.

- Mark all nodes unvisited. Create a set of all the unvisited nodes called the unvisited set, in our case we are going to use a set for visited nodes, not for unvisited nodes.
- Assign to every node a tentative distance value: set it to zero for our initial node. Set the initial node as current.
- For the current node, consider all of its unvisited neighbors and calculate their tentative distances through the current node. Compare the newly calculated tentative distance to the current assigned value and assign the smaller one. For example, if the current node A is marked with a distance of 6, and the edge connecting it with a neighbor B has length 2, then the distance to B through A will

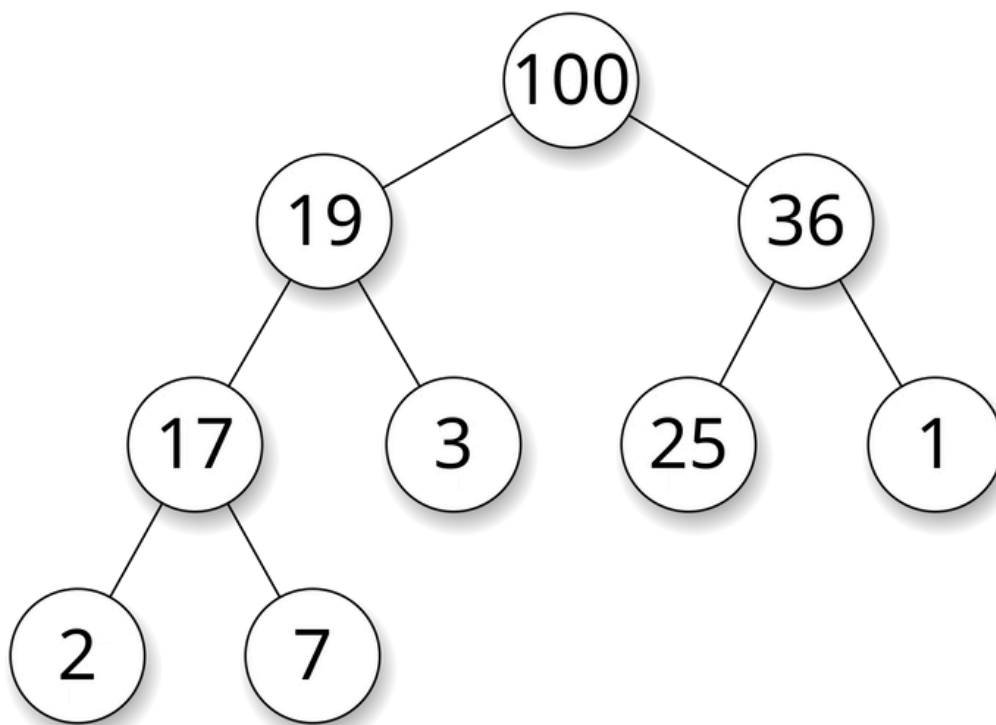
be $6 + 2 = 8$. If B was previously marked with a distance greater than 8 then change it to 8. Otherwise, keep the current value.

- When we are done considering all of the unvisited neighbors of the current node, mark the current node as visited. A visited node will never be checked again.
- Select next unvisited node that is marked with the smallest tentative distance, set it as the new "current node", and go back to step 3.

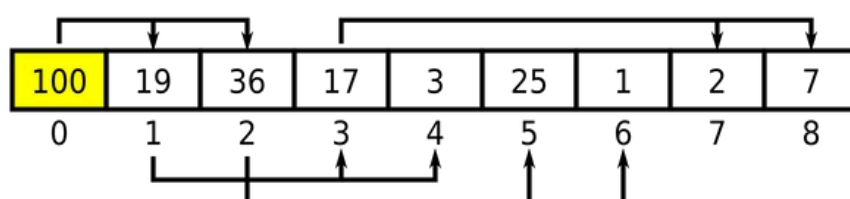
[Wikipedia](#)

What is Heap?

Tree representation



Array representation



In computer science, a heap is a specialized tree-based data structure which is essentially an almost complete tree that satisfies the heap property: in a max heap, for any given node C , if P is a parent node of C , then the key (the value) of P is greater than or equal to the key of C . In a min heap, the key of P is less than or equal to the key of C . The node at the "top" of the heap (with no parents) is called the root node.

[Wikipedia](#)

A heap can be thought of as a priority queue; the most important node will always be at the top, and when removed, its replacement will be the most important. This can be useful when coding algorithms that require certain things to be processed in a complete order, but when you don't want to perform a full sort or need to know anything about the rest of the nodes. For instance, a well-known algorithm for finding the shortest distance between nodes in a graph, Dijkstra's Algorithm, can be optimized by using a priority queue.

[Cprogramming](#)

Why?

I am trying to learn about graphs and its algorithms, because of that I try to read and learn about this in my free time, I recently watched a video about one implementation of Dijkstra in python using a heap was interesting, so I decided to do the same but with go.

I know that there are many articles about the same topic and these articles explain very well what is Dijkstra or what is a heap, this article will be a short article just focus on the implementation, I want to show you a very simple implementation of Dijkstra using heap in Golang.

If you want to read more about Dijkstra you should read this article that I found is amazing.

[An excelent article](#)

Implementation

Dijkstra is an algorithm for searching the short path between two nodes, visiting the neighbors of each node and calculating the cost and the path from origin node keeping always the smallest value, for that we can use a min-heap to keep the min value in each iteration, using push and pop operation, both operations are $O(\log n)$.

First, we need to implement for min-heap, golang has a package in its standard library for that.

the Package heap provides heap operations for any type that implements heap.Interface. A heap is a tree with the property that each node is the minimum-valued node in its subtree.

Godoc - heap

heap.go

```
package main

import hp "container/heap"

type path struct {
    value int
    nodes []string
}

type minPath []path

func (h minPath) Len() int           { return len(h) }
func (h minPath) Less(i, j int) bool { return h[i].value < h[j].value }
func (h minPath) Swap(i, j int)      { h[i], h[j] = h[j], h[i] }

func (h *minPath) Push(x interface{}) {
    *h = append(*h, x.(path))
}

func (h *minPath) Pop() interface{} {
    old := *h
    n := len(old)
    x := old[n-1]
    *h = old[0 : n-1]
    return x
}

type heap struct {
    values *minPath
}

func newHeap() *heap {
    return &heap{values: &minPath{}}
}
```

```
func (h *heap) push(p path) {
    hp.Push(h.values, p)
}

func (h *heap) pop() path {
    i := hp.Pop(h.values)
    return i.(path)
}
```

Secondly, we need to implement the logic for the graph, for that, we use a struct that contains a map to keep the edges among the nodes, with functions to add the edges and get all edges from one node.

The function getPath implement the Dijkstra algorithm to get the shortest path between origin and destiny.

graph.go

```
package main

type edge struct {
    node    string
    weight  int
}

type graph struct {
    nodes map[string][]edge
}

func newGraph() *graph {
    return &graph{nodes: make(map[string][]edge)}
}

func (g *graph) addEdge(origin, destiny string, weight int) {
    g.nodes[origin] = append(g.nodes[origin], edge{node: destiny, weight: weight})
    g.nodes[destiny] = append(g.nodes[destiny], edge{node: origin, weight: weight})
}

func (g *graph) getEdges(node string) []edge {
```

```

    return g.nodes[node]
}

func (g *graph) getPath(origin, destiny string) (int, []string) {
    h := newHeap()
    h.push(path{value: 0, nodes: []string{origin}})
    visited := make(map[string]bool)

    for len(*h.values) > 0 {
        // Find the nearest yet to visit node
        p := h.pop()
        node := p.nodes[len(p.nodes)-1]

        if visited[node] {
            continue
        }

        if node == destiny {
            return p.value, p.nodes
        }

        for _, e := range g.getEdges(node) {
            if !visited[e.node] {
                // We calculate the total spent so far plus the cost and the
                h.push(path{value: p.value + e.weight, nodes: append([]string{
                    node,
                }, p.nodes...)})
            }
        }

        visited[node] = true
    }

    return 0, nil
}

```

main.go

```

package main

import (
    "fmt"

```

```
)  
  
func main() {  
    fmt.Println("Dijkstra")  
    // Example  
    graph := newGraph()  
    graph.addEdge("S", "B", 4)  
    graph.addEdge("S", "C", 2)  
    graph.addEdge("B", "C", 1)  
    graph.addEdge("B", "D", 5)  
    graph.addEdge("C", "D", 8)  
    graph.addEdge("C", "E", 10)  
    graph.addEdge("D", "E", 2)  
    graph.addEdge("D", "T", 6)  
    graph.addEdge("E", "T", 2)  
    fmt.Println(graph.getPath("S", "T"))  
}
```

```
$ go run .  
Dijkstra  
12 [S C B D E T]
```

Github



The DEV Team

PROMOTED





What is an LLM actually doing when it's "thinking"?

Ever wondered what an LLM is doing when it's "thinking"?

In this episode of Release Notes Explained, we cover the fundamentals of how

[Read More](#)

Top comments (4)



Curtis Fenner • Jul 15 '19 • Edited



I don't see the definition of `wasVisited`. However, it seems like you should use a `map[string]bool` instead of `[]string` for your `visited` set so that you don't have to walk over the list.

If you're walking over the list of visited nodes for every edge, your implementation runs in time proportional to $O(|V|*|E|)$! (which defeats the point of using a heap)

You can also simplify your `getPath` implementation. There's no reason to treat the source node specially. You can instead initialize the `visited` set to empty and the heap with just the source, to be visited first. (Notice how the beginning of the function duplicates the contents of the loop!)



Douglas Makey Mendez Molero • Jul 15 '19



for this reason I love to share because is good for me, to improve and learn, at the moment that I wrote the code I was focus in other things and ignore other, thanks master you are 1000% right. <3



Ajinkya Borade • Nov 1 '19



this is still over my head :(



rishabh625 • Aug 28 '21



Checkout this algorithm using min priority queue, please follow me on medium if u like
medium.com/@rishabhmishra131/golan...

[Code of Conduct](#) • [Report abuse](#)



Auth0 PROMOTED

